



SIMULATION (MAA313)

PROJECT, 4.5 ECTS CREDITS

DATE:

MAY 28, 2026

Barrier Option Pricing with Variance Reduction via Importance Sampling

Author:

Paolo Nardi

Instructor:

Daniel Andrén

DIVISION OF MATHEMATICS AND PHYSICS
MÄLARDALEN UNIVERSITY
SE-721 23, VÄSTERÅS, SWEDEN

Abstract

This work investigates the pricing of up-and-in put barrier options using Monte Carlo simulation with importance sampling as a variance reduction technique. We first derive the closed-form solution under the Black-Scholes framework and apply the Broadie-Glasserman-Kou continuity correction to account for discrete barrier monitoring. We then introduce an importance sampling scheme based on a two-phase drift shifting strategy: an upward drift tilt to increase the probability of hitting the barrier, followed by a downward tilt to favor in-the-money terminal prices. The likelihood ratio correction is derived via Girsanov's theorem to preserve unbiasedness. Numerical results confirm that the importance sampling estimator achieves a significantly lower standard error than the standard Monte Carlo estimator, with a roughly sixfold reduction, while more than doubling the fraction of knock-in paths, demonstrating the efficiency of the proposed approach for rare-event pricing of barrier options.

Contents

1	Introduction	3
2	Up-and-In Put Option	3
3	Model Setup	3
3.1	Closed form solution	4
3.2	Broadie-Glasserman-Kou Correction	5
4	Variance Reduction	5
4.1	Importance Sampling	6
4.1.1	Likelihood Ratio Correction	6
4.1.2	Importance Sampling Estimator	7
5	Numerical Implementation	7
6	Results	8
7	Convergence and Confidence Interval	10
8	Conclusions	11
A	Python C0de	12

1 Introduction

The payoff of a conventional option depends only on the price of the underlying relative to the strike at exercise time, by the way there exists so called *path-dependent* options whose payoff depends somehow on all the history of the underlying price. One important class of this type is *barrier options*.

A barrier option is a derivative product that either becomes worthless, must be exercised, or comes into existence if the underlying asset price reaches a certain level during a certain period of times. There are two general classes of barrier options: *knock-in* and *knock-out* options. With the firsts the buyer gets an option that becomes active if and when the underlying hit a barrier level, instead a knock-out option is active from the beginning, but becomes inactive if the stock price hits the barrier value.

For standard European barrier options, closed-form mathematical formulas based on the extended Black-Scholes model exists. While these formulas are derived under the assumption of continuous monitoring of the barrier, most of the real contracts specify discrete monitoring instants. In this context Monte Carlo simulation becomes the most natural approach for pricing these type of instruments. However, the standard Monte Carlo estimator suffers from high variance. In order to mitigate this problem a powerful technique is represented by *importance sampling*.

2 Up-and-In Put Option

In our dissertation we will deal with a knock-in type barrier option, more precisely with an up-and-in put option. However, we can obtain quite similar results with all the other types of barrier options using the same arguments.

An option is called a *up-and-in put* option if it is actually a vanilla put option when S touches an upper barrier B_u , and it expires worthless if the underlying never reaches the barrier level during its life.

3 Model Setup

We assume that the underlying asset follows geometric Brownian motion under the risk neutral measure:

$$\frac{dS_t}{S_t} = rdt + \sigma dW_t,$$

where r , σ are constants and W_t represents a Wiener process.

Let $0 = t_0 < t_1 < \dots < t_N = T$ with equal spacing $\Delta t = T/N$, a time partitioning, and p_i be the value of an up-and-in European put option.

As an option, $p_i(S, t)$ satisfies the Black-Scholes equation for $S < B_u$. If S is always less than B_u its terminal condition should be

$$p_i(S, T) = 0, \quad \text{for } S < B_u.$$

If the asset price S ever touches the upper barrier level by expiry, then the option immediately turns into a vanilla put and must have the identical value as the correspondent vanilla put, so the boundary condition is

$$p_i(B_u, t) = p(B_u, t),$$

where $p(S, t)$ represents the value of a vanilla put option.

Therefore, the fair value of such an option should be the solution of the following problem

$$\begin{cases} \frac{\partial p_i}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 p_i}{\partial S^2} + (r - D_0)S \frac{\partial p_i}{\partial S} - r p_i = 0, & S < B_u, \quad t \leq T, \\ p_i(S, T) = 0, & S < B_u, \\ p_i(B_u, t) = p(B_u, t), & t \leq T. \end{cases}$$

3.1 Closed form solution

In 1991 Rubenstein and Reimer summarized in a comprehensive article the closed form solutions for all the types of barrier option. Their objective is to value a variety of barrier options in the Black-Scholes framework, that is, one in which the underlying asset can be assumed to follow a lognormal random walk and in which arbitrage arguments allow the use of a risk neutral valuation approach. Then discount the expected payoff of the option at expiration by the riskless rate. To do this they need the density of the natural logarithm of the risk neutral underlying asset return, u :

$$f(u) = \left(\frac{1}{\sigma\sqrt{2\pi t}} \right) e^{-\frac{1}{2}v^2}, \quad \text{with } v = \frac{u - \mu t}{\sigma\sqrt{t}}, \quad \mu = \ln \frac{r}{d} - \frac{1}{2}\sigma^2.$$

They also need the density of the natural logarithm of the underlying asset return when the underlying asset price breaches the barrier but ends up at expiration below the barrier, given the underlying asset price starts at $S < B_u$, which is

$$g(u) = e^{\frac{2\mu\alpha}{\sigma^2}} \left(\frac{1}{\sigma\sqrt{2\pi t}} \right) e^{-\frac{1}{2}v^2}, \quad \text{with } v = \frac{u + 2\alpha + \mu t}{\sigma\sqrt{t}}, \quad \alpha = \ln \frac{B_u}{S}.$$

Here we present the formula to price an up-and-in put option. Let K be the strike price, B_u the upper barrier, the fair value of an up-and-in European put option is

$$p_i(S, t) = -S \left(\frac{B_u}{S} \right)^{2\mu+1} \mathcal{N}(-y_1) + K e^{-rT} \left(\frac{B_u}{S} \right)^{2\mu} \mathcal{N}(-y_1 + \sigma\sqrt{T}),$$

$$\mu = \frac{r - \frac{\sigma^2}{2}}{\sigma^2}, \quad y_1 = \frac{\ln \left(\frac{B_u^2}{SK} \right)}{\sigma\sqrt{T}} + (1 + \mu)\sigma\sqrt{T}.$$

3.2 Broadie-Glasserman-Kou Correction

Like we said before, one of the biggest limitations of the closed-form solution we found above, is the assumption that we are able to continuously monitoring the possible breaching of the barrier. However, in the most of the case the control of the barrier can happen only at specified times, and numerical examples (Chance, 1994) indicate that there can be substantial price differences between discrete and continuous barrier options.

In 1997, Broadie, Glasserman and Kou introduced a simple continuity correction for approximate pricing of discrete barrier options. Their method continues to use formulas for continuous barrier options but shifts the barrier to correct for discrete monitoring. In particular, it results that

$$V_m(B_u) = V \left(B_u e^{\beta\sigma\sqrt{T/m}} \right) + o \left(\frac{1}{\sqrt{m}} \right),$$

where V_m is the value of discrete barrier option, V the value of continuous barrier option, m the monitoring times and $\beta \approx 0.5826$.

4 Variance Reduction

It is well known that the standard Monte Carlo estimator can suffer from high variance, particularly for path dependent options, such as the barrier options.

Reducing variance, not only stabilize our result, but it also allows us to reach a good result with a lot less simulation than in a standard Monte Carlo.

The variance reduction technique we choose is the *importance sampling*. In fact, the value of a up-and-in put option is different from zero if the underlying stock price increases and touches the barrier level prior maturity and then decreases under the strike price. We can say that such an event is quite rare, so the idea is to use importance sampling to change the probability distribution in order to make this type of event more probable and study the behavior in this tail condition.

4.1 Importance Sampling

In discrete time with step $\Delta t = T/N$, we can simulate the price process as:

$$\log S_{t+1} - \log S_t = \underbrace{\left(r - \frac{1}{2}\sigma^2\right) \Delta t}_{\mu} + \sigma \sqrt{\Delta t} Z_t, \quad Z_t \stackrel{iid}{\sim} \mathcal{N}(0, 1).$$

Define the log-price process $X_t = \log(S_t/S_0)$, the log-barrier $b = \log(B_u/S_0)$, and $c = \log(S_0/K)$.

We split each simulated path into two phases:

1. **Phase 1 (pre-knock-in):** Drift is shifted upward so that hitting the upper barrier is more probable.
2. **Phase 2 (post-knock-in):** Once the barrier is crossed at time τ , the drift is shifted downward to push the terminal price below the strike K , increasing the probability of a positive put payoff.

The shifted increments are:

$$\Delta X_t = \begin{cases} (\mu + \sigma^2 \Delta t \theta_+) + \sigma \sqrt{\Delta t} Z_t, & t < \tau \quad (\text{Phase 1}), \\ (\mu + \sigma^2 \Delta t \theta_-) + \sigma \sqrt{\Delta t} Z_t, & t \geq \tau \quad (\text{Phase 2}), \end{cases}$$

where the optimal shift parameters are:

$$\theta_+ = \frac{1}{2} - \frac{r}{\sigma^2} + \frac{2b+c}{T\sigma^2}, \quad \theta_- = \frac{1}{2} - \frac{r}{\sigma^2} - \frac{2b+c}{T\sigma^2}.$$

The parameter θ_+ tilts paths towards the barrier, while θ_- tilts the post-barrier path towards the in-the-money region for the put.

4.1.1 Likelihood Ratio Correction

Each shifted increment corresponds to replacing the original drift μ with a modified drift $\mu + \sigma^2 \Delta t \theta$. By Girsanov's theorem, the log-likelihood ratio (log-LR) between the original measure $\mathbb{Q}$ and the importance sampling measure $\tilde{\mathbb{Q}}$ accumulates over the path. The log-LR for a path that knocks in at step τ with log-return X_τ and terminates with log-return $X_T = L_m$ is:

$$\log \mathcal{L} = (\theta_- - \theta_+) X_\tau - \theta_- X_T + n \psi_-,$$

where the cumulant generating function correction is:

$$\psi_- = \mu \theta_- + \frac{1}{2} \sigma^2 \Delta t \theta_-^2.$$

The likelihood ratio is then $\mathcal{L} = e^{\log \mathcal{L}}$.

4.1.2 Importance Sampling Estimator

The IS price estimator for the up-and-in put is:

$$\hat{V}^{\text{IS}} = \frac{1}{m} \sum_{i=1}^m e^{-rT} \max(K - S_T^{(i)}, 0) \mathbf{1}_{\{\tau^{(i)} < \infty\}} \mathcal{L}^{(i)},$$

where m is the number of simulated paths and $S_T^{(i)}$ is the terminal stock price under the importance sampling measure $\tilde{\mathbb{Q}}$.

The standard error of the estimator is:

$$\widehat{\text{SE}} = \frac{s}{\sqrt{m}}, \quad s^2 = \frac{1}{m-1} \sum_{i=1}^m \left(\hat{V}_i - \hat{V}^{\text{IS}} \right)^2.$$

As a result, the importance sampling estimator achieves substantial variance reduction.

5 Numerical Implementation

The implementation is written in Python and is structured into three main components.

- **up_and_in_put_price**

Computes the analytical price of the up-and-in put european option. This function implements the closed form solution derived in Section 3.1.

- **broadie_glasserman_kou_correction**

Computes the price of the up-and-in european put option considering a discrete monitoring of the barrier, adding the correction we presented in Section 3.2.

- **simulate_payoffs**

Generates Monte Carlo sample paths of the underlying asset using a simulation of geometric Brownian motion. For each path it computes the payoff and applies discounting.

- **simulate_payoffs_importance_sampling**

It computes the optimal shift parameters, and it builds the shifted increments. After it has computed the log-likelihood ratio, the function returns the simulated payoffs and finds the price estimator. It follows the procedure explained in Section 4.

- main

Outputs numerical and graphical results.

6 Results

We considered the parameter set $S_0 = 100$, $K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$, $B_u = 120$, and $n = 252$ observation points.

Table 1 reports the analytical price with and without Broadie-Glasserman-Kou correction, the estimated price via standard Monte Carlo simulation, its standard error with relative 95% confidence interval, and the percentage of paths that hit the barrier.

Table 2 shows the analytical price with and without Broadie-Glasserman-Kou correction, the estimated price via Monte Carlo simulation and Importance Sampling variance reduction technique. Even in this case are also visible the new standard error with relative 95% confidence interval, and the percentage of paths that hit the barrier.

From the comparison of the two tables we can see that in both the cases the estimated prices are a little bit far from the analytical price, but this is reasonable with the limitation of the Black-Scholes model relative to the continuous monitoring of the barrier. In fact, we see that the simulated prices converges to the analytical price with the Broadie-Glasserman-Kou correction. The estimator for up-and-in put option and the importance sampling produce nearly identical price estimates, confirming that the importance sampling technique is unbiased. However, the most important feature to notice is the fact that the standard error of the importance sampling estimator is significantly lower than the standard Monte Carlo estimator. Moreover, in the importance sampling simulation the percentage of knocked in paths more than doubled with respect to the one of standard Monte Carlo, confirming the efficiency of importance sampling.

The reduction in variance implies that we can reach the same accuracy in the price of the option with a lot less simulation. For this reason, thanks to our analysis we can state the efficiency of importance sampling approach in pricing barrier options.

In Figure 1 we can see the graphical representation of some of the simulated paths, in red are shown the one which are knocked in.

Table 1: Simulation Results

Metric	Value
Analytical price	0.2134
Corrected price (Broadie–Glasserman–Kou)	0.1774
Estimated price	0.1740
Standard error	0.0063
95% Confidence Interval	[0.1617, 0.1863]
Percentage of paths that hit the barrier	39.20%

Table 2: Importance Sampling Results

Metric	Value
Analytical price	0.2134
Corrected price (Broadie–Glasserman–Kou)	0.1774
Estimated price	0.1788
Standard error	0.00098
95% Confidence Interval	[0.1769, 0.1807]
Percentage of paths that hit the barrier	89.81%

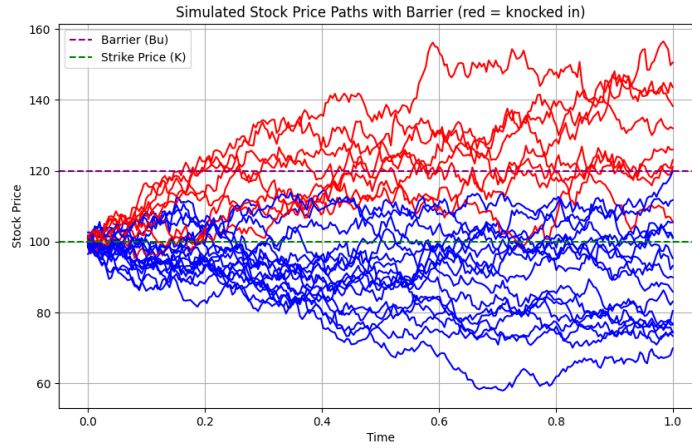


Figure 1: Simulated Paths, (red=knocked in paths)

7 Convergence and Confidence Interval

Figure 2 shows the convergence of both estimators with 95% confidence intervals. The importance sampling estimator exhibits significantly narrower confidence bands, demonstrating its superior efficiency.

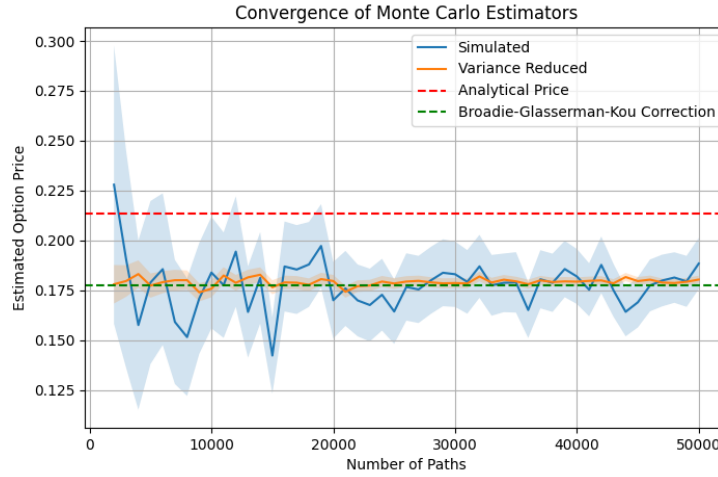


Figure 2: Monte Carlo convergence with 95% confidence intervals for different number of sample paths

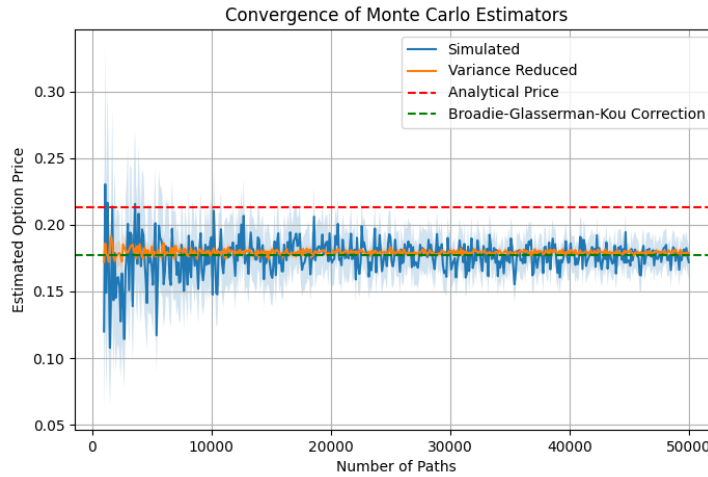


Figure 3: Monte Carlo convergence with 95% confidence intervals for different number of sample paths with a finer intervals discretization

8 Conclusions

The importance sampling represents a solid and efficient variance reduction technique for pricing barrier options. Both numerically and graphically we saw that this method has significantly reduced the variance of estimator allowing us to reach the same accuracy of a standard Monte Carlo estimator with less simulated paths, reducing considerably the computational cost.

A Python C0de

```
1 #UP-AND-IN PUT OPTION PRICING WITH MONTE CARLO SIMULATION AND
  IMPORTANCE SAMPLING
2
3 #In the next lines we will use the Monte Carlo method to
  estimate the value of an exotic option, in particular an
  up-and-in put option.
4 #We know that the option expires worthless if the stock's
  price doesn't reach the barrier level during the all life
  of the option, and
5 #that if the barrier is reached, the payoff results in the
  payoff of a vanilla put option.
6
7 import numpy as np
8 import matplotlib.pyplot as plt
9 from scipy.stats import norm
10
11 #Legend:
12 # S0: initial stock price
13 # K: strike price
14 # r: risk free interest rate
15 # sigma: volatility of the stock
16 # T: maturity
17 # Bu: upper barrier level
18 # n: number of steps
19 # m: number of paths
20
21 #Here we implement the analytical closed form solution for an
  up and in put option
22 def up_and_in_put_price(S0, K, r, sigma, T, Bu):
23
24     mu=(r-0.5*sigma**2)/(sigma**2)
25     y_1=(np.log(Bu**2/(S0*K)))/(sigma*np.sqrt(T))+((mu+1)*
  sigma*np.sqrt(T))
26
27     A_1=-S0*(Bu/S0)**(2*(mu+1))*norm.cdf(-y_1)
28     A_2=K*np.exp(-r*T)*(Bu/S0)**(2*mu)*norm.cdf(-y_1+sigma*np
  .sqrt(T))
29
30     value=A_1+A_2
31     return value
32
33 #Implement here the Broadie-Glasserman-Kou correction to
  estimate the price of the up-and-in put option
34 def broadie_glasserman_kou_correction(S0, K, r, sigma, T, Bu,
  n, beta):
35     dt=T/n
36     barrier_term=np.exp(beta*sigma*np.sqrt(dt))
```

```

37     corrected_value=up_and_in_put_price(S0, K, r, sigma, T,
38     Bu*barrier_term)
39     return corrected_value
40 #Monte Carlo simulation
41 def simulate_payoffs(S0, K, r, sigma, T, Bu, n, m):
42     dt=T/n
43     drift=(r-0.5*sigma**2)*dt
44     vol=sigma*np.sqrt(dt)
45
46     dlogS=np.random.normal(drift, vol, size=(m,n))
47     logS=np.log(S0)+np.cumsum(dlogS, axis=1)
48     S=np.exp(logS)
49
50     knocked_in=np.any(S>=Bu, axis=1) #boolean array which
51     indicate if the barrier was reached or not
52     S_T=S[:,-1]
53     payoffs=np.where(knocked_in, np.maximum(K-S_T, 0.0), 0.0)
54     #np.where (x, y, z) returns y if x is true, otherwise z
55     discount=np.exp(-r*T)
56     discounted_payoffs=discount*payoffs
57
58     price_estimation=np.mean(discounted_payoffs)
59     std_error=discounted_payoffs.std(ddof=1)/np.sqrt(m)
60     low_CI=price_estimation-1.96*std_error
61     up_CI=price_estimation+1.96*std_error
62     knocked_in_fraction=np.mean(knocked_in)
63     knocked_in_percentage=knocked_in_fraction*100
64
65     return{
66         "price_estimation": price_estimation,
67         "std_error": std_error,
68         "paths": S,
69         "low_CI": low_CI,
70         "up_CI": up_CI,
71         "knocked_in_percentage": knocked_in_percentage
72     }
73 #Monte Carlo simulation with importance sampling
74 def simulate_payoffs_importance_sampling(S0, K, r, sigma, T,
75     Bu, n, m):
76     dt=T/n
77     drift=(r-0.5*sigma**2)*dt
78     vol=sigma*np.sqrt(dt)
79
80     b = np.log(Bu / S0)
81     c = np.log(S0 / K)

```

```

82     theta_plus=0.5- r/(sigma**2) + (2*b+c)/(T*sigma**2)
83     theta_minus=0.5- r/(sigma**2) - (2*b+c)/(T*sigma**2)
84
85     psi_minus = drift * theta_minus + 0.5 * sigma**2 * dt *
theta_minus**2
86
87     payoffs = np.zeros(m)
88     knocked_in = np.zeros(m, dtype=bool)
89     for i in range(m):
90         Z = np.random.standard_normal(n)
91         log_S = np.log(S0)
92         tau = None
93         L_tau = 0.0 # cumulative log-return at knock-in
94         L_m = 0.0 # cumulative log-return at expiry
95
96         for t in range(n):
97             if tau is None:
98                 increment = (drift + sigma**2 * dt *
theta_plus) + vol * Z[t] # phase 1
99             else:
100                 increment = (drift + sigma**2 * dt *
theta_minus) + vol * Z[t] # phase 2
101
102                 log_S += increment
103                 L_m = log_S - np.log(S0) # log(S_t / S0)
104
105                 if tau is None and L_m >= b: # barrier hit
106                     tau = t
107                     L_tau = L_m
108
109         S_T = np.exp(log_S)
110         knocked_in[i] = (tau is not None)
111
112
113         if tau is not None:
114             put_payoff = max(K - S_T, 0.0)
115             if put_payoff > 0:
116                 log_LR = ((theta_minus - theta_plus) * L_tau
- theta_minus * L_m
117                     + n * psi_minus)
118                 LR = np.exp(log_LR)
119                 payoffs[i] = np.exp(-r * T) * put_payoff * LR
120
121
122
123     payoffs_IS=payoffs
124     price_estimation_IS=np.mean(payoffs_IS)
125     std_error_IS=payoffs_IS.std(ddof=1)/np.sqrt(m)
126     low_CI_IS=price_estimation_IS-1.96*std_error_IS
127     up_CI_IS=price_estimation_IS+1.96*std_error_IS

```

```

128     knocked_in_fraction_IS=np.mean(knocked_in)
129     knocked_in_percentage_IS=knocked_in_fraction_IS*100
130
131     return{
132         "price_estimation_IS": price_estimation_IS,
133         "std_error_IS": std_error_IS,
134         "low_CI_IS": low_CI_IS,
135         "up_CI_IS": up_CI_IS,
136         "knocked_in_percentage_IS": knocked_in_percentage_IS
137     }
138
139
140 def main():
141     S0=100
142     K=100
143     r=0.05
144     sigma=0.2
145     T=1
146     Bu=120
147     n=252
148     m=10000
149     beta=0.5826 #Broadie-Glasserman-Kou correction parameter
150
151     analytical_price = up_and_in_put_price(S0, K, r, sigma, T
152     , Bu)
153     corrected_price = broadie_glasserman_kou_correction(S0, K
154     , r, sigma, T, Bu, n, beta)
155     result = simulate_payoffs(S0, K, r, sigma, T, Bu, n, m)
156     result_IS = simulate_payoffs_importance_sampling(S0, K, r
157     , sigma, T, Bu, n, m)
158     price = result["price_estimation"]
159     price_IS = result_IS["price_estimation_IS"]
160     error = result["std_error"]
161     error_IS = result_IS["std_error_IS"]
162     S = result["paths"]
163     low_CI = result["low_CI"]
164     low_CI_IS = result_IS["low_CI_IS"]
165     up_CI = result["up_CI"]
166     up_CI_IS = result_IS["up_CI_IS"]
167     knocked_in_percentage = result["knocked_in_percentage"]
168     knocked_in_percentage_IS = result_IS["
knocked_in_percentage_IS"]
169
170     #Convergence study
171     path_sizes=np.arange(1000, m+1, 100)
172     simulated_mean, variancereduced_mean, =[] , []
173     se_simulated, se_variancereduced = [], []
174     for size in path_sizes:

```

```

172     res_simulated = simulate_payoffs(S0, K, r, sigma, T,
    Bu, n, size)
173     res_variancereduced =
    simulate_payoffs_importance_sampling(S0, K, r, sigma, T,
    Bu, n, size)
174     simulated_mean.append(res_simulated["price_estimation
    "])
175     variancereduced_mean.append(res_variancereduced["
    price_estimation_IS"])
176     se_simulated.append(res_simulated["std_error"])
177     se_variancereduced.append(res_variancereduced["
    std_error_IS"])
178
179     #Convergence plot with confidence intervals
180     z=1.96
181     fig, ax = plt.subplots(figsize=(8, 5))
182     for means, se, label in [ (simulated_mean, se_simulated,
    "Simulated"), (variancereduced_mean, se_variancereduced, "
    Variance Reduced") ]:
183         ax.plot(path_sizes, means, label=label)
184         ax.fill_between(path_sizes, np.array(means) - z * np.
    array(se), np.array(means) + z * np.array(se), alpha=0.2)
185         ax.axhline(y=analytical_price, color='red', linestyle='--
    ', label='Analytical Price')
186         ax.axhline(y=corrected_price, color='green', linestyle='
    --', label='Broadie-Glasserman-Kou Correction')
187         ax.set_xlabel('Number of Paths')
188         ax.set_ylabel('Estimated Option Price')
189         ax.set_title('Convergence of Monte Carlo Estimators')
190         ax.legend()
191         ax.grid(True)
192         plt.show()
193
194     print("----- Simulation Results
    -----")
195     print(f"Analytical price: {analytical_price:.4f}")
196     print(f"Corrected price (Broadie-Glasserman-Kou): {
    corrected_price:.4f}")
197     print(f"Estimated price: {price:.4f}, Standard error: {
    error:.4f}")
198     print(f"95% Confidence Interval: [{low_CI:.4f}, {up_CI:.4
    f}]")
199     print(f"Percentage of paths that hit the barrier: {
    knocked_in_percentage:.2f}%")
200     print("
    -----")
201

```

```

202     print("----- Importance Sampling Results
-----")
203     print(f"Analytical price: {analytical_price:.4f}")
204     print(f"Corrected price (Broadie-Glasserman-Kou): {
corrected_price:.4f}")
205     print(f"Estimated price: {price_IS:.4f}, Standard error:
{error_IS:.5f}")
206     print(f"95% Confidence Interval: [{low_CI_IS:.4f}, {
up_CI_IS:.4f}]")
207     print(f"Percentage of paths that hit the barrier: {
knocked_in_percentage_IS:.2f}%")
208     print("
-----")
209
210
211
212     #We can also plot some of the simulated paths to
visualize the behavior of the stock price and the barrier
level.
213     num_paths_to_plot = 25 # Plot 25 paths for visualization
214     t = np.linspace(0, T, n) # Time points from 0 to T
215
216     plt.figure(figsize=(10, 6))
217     for i in range(num_paths_to_plot):
218         color = 'red' if np.any(S[i] >= Bu) else 'blue' #
Red for paths that hit the barrier, blue otherwise
219         plt.plot(t, S[i], color=color)
220         plt.axhline(y=Bu, color='purple', linestyle='--', label='
Barrier (Bu)')
221         plt.axhline(y=K, color='green', linestyle='--', label='
Strike Price (K)')
222         plt.xlabel('Time')
223         plt.ylabel('Stock Price')
224         plt.title('Simulated Stock Price Paths with Barrier (red
= knocked in)')
225         plt.legend()
226         plt.grid(True)
227         plt.show()
228
229
230 main()

```

References

Y. ZHU, X. WU, I. CHERN, Z. SUN, Derivative Securities and Difference Methods, Springer Finance, 2nd edition (2013).

M. BROADIE, P. GLASSERMAN, S. KOU, A Continuity Correction for Discrete Barrier Options, Mathematical Finance, Vol. 7, No. 4 (October 1997), 325-348.

M. RUBINSTEIN, E. REINER, BREAKING DOWN THE BARRIER, Risk Magazine, VOL 4/NO 8/ SEPTEMBER 1991, 28-35.

J. RÖMAN, Analytical Finance, Palgrave McMillan, Vol. 1, 2017.